SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool *(BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 3 **Total Marks:50**

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I Gowtham M 192373060(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

ERROR ANALYSIS TASK

Q1. The following program is intended to perform simple arithmetic operations, function calls, array access, and looping constructs. However, the code contains **multiple errors introduced at different phases of compilation**, including **lexical, syntax, semantic, logical, and runtime errors**.

(a) Error Identification

The given C program contains syntax, semantic, logical and runtime errors.

Line No.	**Error**	**Description**
51	i <= n	Causes access beyond the array boundary when i == n.
58	Missing semicolon	A semicolon is required after the array declaration.
61	sumArray(arr)	The function requires two arguments, but only one is passed.
64	Missing semicolon	The printf() statement must end with a semicolon.
68	a / b	Division by zero occurs because b = 0.

70– 71	*p = 20	Pointer p is uninitialized and does not point to a valid object.

(b) Classification of Errors

1. Lexical Error

A lexical error occurs when the source code contains an invalid token or illegal character. No clear lexical error is present in the given Q1 program.

2. Syntax Error

A syntax error occurs when the rules of C grammar are violated. For example, the missing semicolons in lines 58 and 64 cause syntax errors.

3. Semantic Error

A semantic error occurs when a statement violates the meaning or requirements of the language. In line 61, the function sumArray() requires two arguments, but only one argument is provided.

4. Logical Error

A logical error occurs when the program's logic is incorrect. The loop condition i <= n should be i < n to prevent accessing an invalid array index.

5. Runtime Error

A runtime error occurs during program execution. Division by zero and dereferencing an uninitialized pointer can cause undefined behaviour.

(b) Compilation Behaviour

1. Syntax errors, such as missing semicolons, are detected during compilation.
2. The incorrect function call may be reported as a compile-time diagnostic.
3. Division by zero and invalid pointer access can cause problems during execution.
4. Array-boundary errors may not be detected by a normal compiler.
5. Early syntax errors can prevent later stages of compilation from correctly analysing the program.

(d) Corrected C Program

```
#include <stdio.h>

int sumArray(int arr[], int n) {
int i, sum = 0;

   for (i = 0; i < n; i++) {
sum = sum + arr[i];
   }

   return sum;
}

int main() {
   int arr[5] = {1, 2, 3, 4, 5};
int total;

   total = sumArray(arr, 5);

   if (total == 15) {
printf("Correct Sum\n");
   }

   int a = 10, b = 2;
int c = a / b;
   printf("Division = %d\n", c);

   int value = 20;
int *p = &value;
   *p = 20;

   return 0;
}
```

Output:

Correct Sum

Division = 5

(e) Impact Analysis

1. Array out-of-bounds access: Accessing an index outside the array may read unwanted memory or cause undefined behaviour.
2. Division by zero: Dividing an integer by zero causes undefined behaviour and may terminate the program.
3. Improper pointer usage: Dereferencing an uninitialized pointer may cause a crash or memory corruption.

(f) Advanced Compiler Improvements

1. Better error messages: Display the line number, error type, cause and suggested correction.
2. Early runtime error detection: Use static analysis and sanitizers to identify potential errors.
3. Improved debugging: Provide breakpoints, variable inspection and stack traces.
4. Error recovery: Allow the compiler to continue analysing the source code after recoverable errors.

Q2.The following C program contains **multiple lexical errors** introduced intentionally. These errors occur due to **invalid tokens, incorrect identifiers, malformed constants, and illegal symbols**, which are typically detected during the **lexical analysis phase** of a compiler.Carefully examine the program and answer the questions given below.

(a) Error Identification

The given program contains invalid identifiers, malformed constants, illegal symbols and incorrectly written literals.

Line No.	**Invalid Token**	**Correct Token**
130	1num	num1
131	rate@	rate

132	'abc'	"abc"
133	50$20	50+20
134	num#1	num1
135	%count	count
137	Missing closing quote	"Welcome to Compiler Design"
139	09	9
140	0xZ12	0x12
141	12.3.4	12.34
143	"Hello	"Hello"
147	invalid-id	invalid_id
149	5..6	5.6
150	10@5	10*5
152	'xy'	"xy"
153	3value	value3

(b) Error Classification

1. Identifier Rules

In C, identifiers can contain letters, digits and underscores, but cannot start with a digit. Special characters such as @, # and % cannot be used in ordinary identifiers.

2. Constant Formation

- 09 is invalid as a C integer constant because it uses an invalid octal digit.
- 0xZ12 contains an invalid hexadecimal digit.
- 12.3.4 contains more than one decimal point.
- 5..6 is not a valid single floating-point constant.

3. String and Character Literals

Strings must begin and end with double quotes. Character literals are normally written using single quotes around one character.

4. Operator and Symbol Rules

The symbols $ and @ cannot be used as ordinary arithmetic operators in these expressions.

(c) Token Correction

Invalid tokens can be corrected by following C identifier, constant and literal formation rules.

Examples:

- 1num → num1
- invalid-id → invalid_id
- 3value → value3
- 0xZ12 → 0x12
- 5..6 → 5.6
-

(d) Compiler Behaviour Analysis

The lexical analyzer reads the source program character by character and groups valid characters into tokens. When it finds an invalid token, it reports a lexical error.

Depending on the compiler, it may stop compilation or continue scanning through error recovery to identify additional errors.

(e) Advanced Thinking

DFA idea: If an identifier begins with a digit and is followed by letters or underscores, the scanner can flag an invalid identifier-like sequence. Otherwise, it processes the digits as a numeric constant.

Improved error messages: The compiler should display the line number, invalid token, reason for the error and a suggested correction.

Q3. The following C program is designed to perform arithmetic operations, function calls, conditional checks, pointer usage, and looping. However, the program contains **multiple errors distributed across different phases of a compiler**, including:

(a) Error Identification

Line No.	**Error**	**Description**
197	Missing semicolon	The return statement needs a semicolon.
202	#value	Invalid identifier.
203	Missing semicolon	The float declaration is incomplete.
206	divide(10)	Two arguments are required, but only one is provided.

208	result = 5	Assignment is used instead of comparison.
209	Missing semicolon	The printf() statement needs a semicolon.
214	x / y	Division by zero because y = 0.
218	arr[4]	Invalid index for an array of four elements.
221–222	*ptr = 25	Uninitialized pointer is dereferenced.
225	+ 0	Redundant expression that can be optimized.
227–229	while (x < 20)	Infinite loop because x is never incremented.

(b) Classification by Compiler Phase 1. Lexical: Invalid identifier #value.

2. Syntax: Missing semicolons.
3. Semantic: Incorrect function argument count.
4. Intermediate code / logic: Assignment instead of comparison may produce unintended control flow.
5. Optimization: The expression + 0 is redundant.
6. Runtime: Division by zero, invalid array access and uninitialized pointer dereferencing.

(c) Compilation vs Execution Compile-time errors:

- Invalid identifier.
- Missing semicolons.
- Incorrect function argument count.

Execution-time errors:

- Division by zero.
- Invalid array access.
- Invalid pointer access.
- Infinite loop.

Syntax errors may prevent the compiler from generating valid intermediate code. Runtime errors can remain undetected until execution unless additional analysis or checking is used.

(d) Corrected C Program

```
#include <stdio.h>
```

```
int divide(int a, int b) {
if (b == 0) {
      printf("Cannot divide by zero\n");
return 0;
   }    return a
/ b;
}

int main() {    int value =
20;    float num = 15.5;
int result = divide(10, 2);

   if (result == 5) {
printf("Result is 5\n");    }


   int x = 10, y = 2;
int z = x / y;
   printf("Division: %d\n", z);

   int arr[4] = {1, 2, 3, 4};
printf("%d\n", arr[3]);

   int *ptr = &value;
   *ptr = 25;

   int total = x + y * result;
printf("Total: %d\n", total);

   while (x < 20) {
printf("%d\n", x);
x++;
   }
```

```
    return 0;
}
```

(e) Deep Analysis

1. Pointer Usage: An uninitialized pointer may point to an invalid memory location. Dereferencing it causes undefined behaviour and may crash the program.
2. Array Bounds: An array containing four elements has valid indexes from 0 to 3. Accessing arr[4] causes undefined behaviour and may access memory outside the array.
3. Operator Precedence: Multiplication has higher precedence than addition. Therefore, x + y * result is evaluated as x + (y * result). The compiler must preserve this order when generating intermediate code.

(f) Advanced Compiler Improvements

1. Runtime error detection: Use static analysis, bounds checking and sanitizers to identify potential errors early.
2. Better debugging messages: Display the source line, error description and suggested correction.
3. Optimization: Remove redundant expressions such as x + 0 when doing so preserves program behaviour.
4. Debugging support: Provide breakpoints, variable inspection and stack traces to make errors easier to locate.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand